Deep Learning Algorithms

Dr. Mahdieh Khalilinezhad

Agenda

- What is Deep Learning?
- Neural Networks Overview
- Activation Functions
- Forward Pass & Loss Calculation
- Introduction to Backpropagation
- Backpropagation in Detail (Math & Concepts)
- Gradient Descent & Optimizers
- Practical Considerations

What is Deep Learning?

- **Definition**: "Deep Learning is a subset of Machine Learning based on artificial neural networks with multiple layers."
- **Key Points**:
  - Mimics the brain’s neural structure.
  - Used for high-dimensional data (images, text, audio).
  - Key areas: computer vision, NLP, speech recognition.

Neural Networks Overview

Components:

- Input Layer
- Hidden Layers
- Output Layer

Explanation:

- Neurons in each layer are connected to the next layer.
- The hidden layers process and extract patterns from the data.

Activation Functions

- **Purpose**: Non-linearity in networks, because the life is not perfect and we have noise always in real world, therefore we add non linearity to the model.
- Types:
  - ReLU: $f(x) = \max(0, x)$
  - Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$
  - Tanh: $f(x) = \tanh(x)$

Forward Pass & Loss Calculation

- **Forward Pass**: Information flows from input to output.
- **Linear Transformation**: $z = W \cdot x + b$
- **Activation Function**: $a = f(z)$
- **Loss Function**:
  - Measures error.
  - Common examples: MSE, Cross-Entropy.

Gradient:

Gradient represent the direction in which the loss function will increase the most, for a small step takes.

The negative of gradient is the direction of (w_0, w_1) space in which the function will decrease most in value, for small step around the point.

We want the loss decrease therefore, negative of gradient is the direction we take.

Introduction to Backpropagation

Definition: "Backpropagation is the algorithm that computes the gradient of the loss function with respect to the weights."

Gradient:

Purpose: Adjust weights to minimize the loss.

Process:

1. Compute forward pass.
2. Compute loss.
3. Propagate the error backward.

Backpropagation in Detail

Step-by-Step:

1. Forward Pass.
2. Loss calculation.
3. Gradient computation using the chain rule.
4. Update weights with gradient descent.

Formula:

$$W_{new} = W_{old} - \eta \cdot \frac{\partial L}{\partial W}$$

Mathematical Formulation of Backpropagation

Key Equition:

- Forward pass: $z^l = W^l a^{l-1} + b^l$
- Loss: $L = \frac{1}{2} \sum (y - \hat{y})^2$
- Backward pass: $\delta^l = \frac{\partial L}{\partial a^l} \cdot f'(z^l)$
- Gradient: $\frac{\partial L}{\partial W^l} = \delta^l \cdot (a^{l-1})^T$

Gradient Descent Optimization

- Optimization algorithm used to minimize the loss function. In the way that weights and bias change for next forward and the loss for next prediction will be reduce.

Types:

- Batch Gradient Descent
- Stochastic Gradient Descent (SGD)
- Mini-batch Gradient Descent

Optimizers

- Common Optimizer
  - SGD: Basic method.
  - Momentum: Adds inertia.
  - RMSprop: Adapts learning rates.
  - Adam: Combines Momentum and RMSprop.

Challenges in Backpropagation

- **Vanishing Gradients**: Gradients become too small, slowing learning.
- **Exploding Gradients**: Gradients become too large, destabilizing learning.
- **Solutions**: Use ReLU, advanced optimizers, batch normalization.

Practical Considerations

- **Hyperparameters**:
  - Learning rate,
  - batch size,
  - epochs,
  - optimizers,
  - Activation functions,
  - number of neurons,
  - number of filters(in Conv2D)

Techniques:

- Dropout, regularization, early stopping.

Summery

- Neural networks learn by adjusting weights using backpropagation.
- Backpropagation uses the chain rule to compute gradients.(with Partial derivative)
- Optimizers like SGD and Adam help reduce loss.

Practical Challenges: Vanishing gradients, overfitting

Deep Learning Problems

- Overfitting
- Underfitting
- Exploding Gradient
- Vanishing Gradients

Definition: Overfitting occurs when a machine learning model learns not only the underlying patterns in the training data but also the noise and specific details. This causes the model to perform very well on the training data but poorly on unseen test data or real-world applications.

Symptoms:

- High accuracy on the training data but significantly lower accuracy on the test data.
- The model is too complex (too many parameters or too many layers), allowing it to "memorize" the training data.
-

Deep Learning Problems

- **Solution:**
  - **Regularization**: Add penalties to complex models (e.g., L2 regularization).
  - **Dropout**: Randomly drop neurons during training to prevent co-adaptation of features.
  - **Cross-Validation**: Use techniques like k-fold cross-validation to check how well the model generalizes to unseen data.
  - **Early Stopping**: Stop training once the model's performance on a validation set stops improving.

2. Underfitting:

Definition: Underfitting occurs when a model is too simple to capture the underlying patterns in the data. It fails to learn adequately from the training data and hence performs poorly on both the training data and the test data.

Symptoms:

- Low accuracy on both training and test data.
- The model is not complex enough to represent the data (e.g., using a linear model on non-linear data).

Underfitting

- Solution:
  - Increase model complexity (add more parameters or more layers in the case of neural networks).
  - Train the model longer (more epochs).
  - Provide more features or use feature engineering to better capture the complexity of the data.

3. Exploiting Gradient

Definition: Exploding gradients occur during the training of deep neural networks when the gradients (which are used to update weights) become excessively large. This causes weight updates to be too large, leading to unstable learning and divergence of the model.

Causes:

- **Deep Networks**: When there are too many layers in a network, small changes in the early layers can explode as they propagate backward during backpropagation.
- **Improper Weight Initialization**: If the weights are initialized too large, it can contribute to gradients becoming excessively large.

Exploiting Gradient

- **Symptoms**:
  a. Loss becomes NaN (not a number) or oscillates wildly during training.
  b. Model fails to converge.
- **Solution**:
  a. **Gradient Clipping**: Limit the maximum gradient values to prevent them from growing too large.
  b. **Weight Initialization**: Use proper initialization methods like Xavier or He initialization.
  c. **Batch Normalization**: Normalize the output of each layer, helping to keep gradients in a reasonable range.

Vanishing Gradient

- **Definition**: Vanishing gradients occur when the gradients become very small, preventing the weights from updating effectively, especially in the earlier layers of deep networks. As a result, the network learns very slowly or not at all.
- **Causes**:
  - **Deep Networks**: In very deep networks, small gradients during backpropagation become even smaller as they propagate backward through multiple layers, causing earlier layers to stop learning.
  - **Activation Functions**: Certain activation functions, like the Sigmoid and Tanh, squash input values to a small range (e.g., [0, 1] for Sigmoid), which can cause gradients to become very small.

Vanishing Gradient

Symptoms:

- The model trains extremely slowly.
- The weights in the earlier layers barely change during training.

Solution:

- **ReLU Activation Function**: ReLU avoids vanishing gradients because its derivative is 1 for positive values, preventing the shrinking of gradients.
- **Batch Normalization**: Helps reduce internal covariate shift and prevents gradients from vanishing.
- **Proper Initialization**: Use initialization methods that avoid too small initial weights (e.g., He initialization).
- **ResNet (Residual Networks)**: Introduce skip connections that allow gradients to bypass some layers, helping gradients propagate backward.

Problem	Solution	Illustration
Overfitting	Regularization, Dropout, Early Stopping	Regularization loss curve, dropout neuron diagram, early stopping validation loss curve
Underfitting	Increase complexity, train for more epochs	Network size comparison, training curves
Exploding Gradients	Gradient clipping, proper initialization, Adam	Gradient value plot, weight initialization comparison
Vanishing Gradients	ReLU, Batch Norm, He Init., Skip Connections (ResNet)	Gradient flow diagrams with and without ReLU, batch norm, and skip connections

Summary

- **Overfitting**: The model is too complex and memorizes the training data, leading to poor generalization.
- **Underfitting**: The model is too simple and cannot learn the underlying patterns in the data.
- **Exploding Gradients**: The gradients become too large, leading to unstable weight updates and divergence.
- **Vanishing Gradients**: The gradients become too small, making it hard for the model to learn and update its weights effectively.